

Java Backend Architecture

Interview Series

Chapter 10.8

Search Data Sync: CDC & Debezium

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

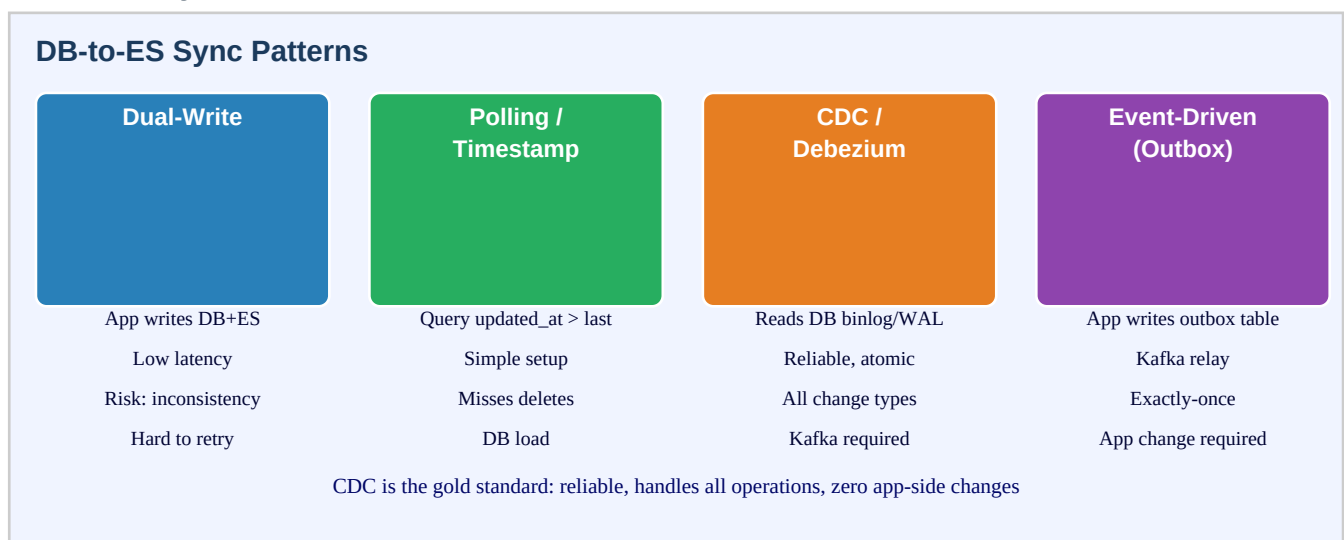
Chapter 10.8 – Search Data Sync: CDC & Debezium

Introduction

Keeping an Elasticsearch index synchronized with a primary relational database is one of the central architectural challenges in search-enabled systems. The primary database remains the system of record; Elasticsearch is a derived read model optimized for search. Any approach to synchronization must handle creates, updates, and deletes reliably — including partial failures, network outages, and application restarts.

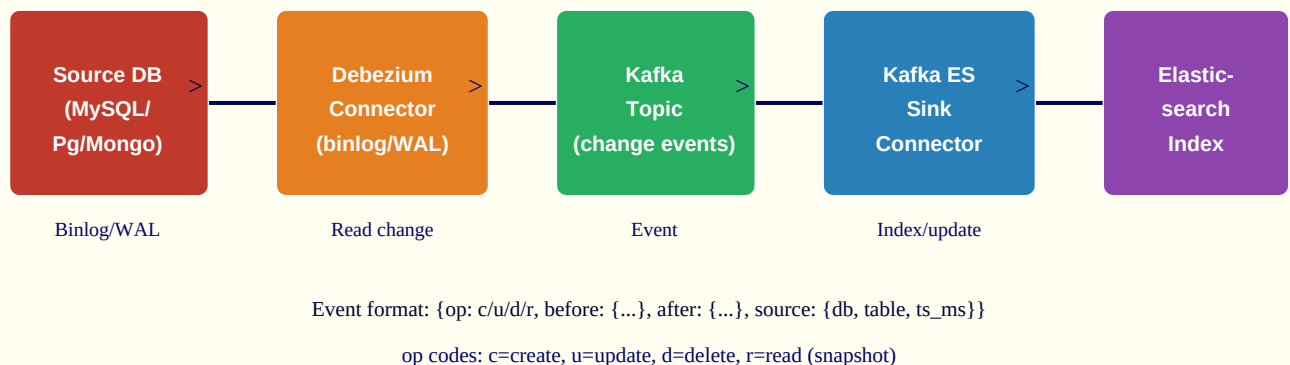
Change Data Capture (CDC) with Debezium has become the industry standard for database-to-ES synchronization. It reads the database transaction log (binlog for MySQL, WAL for PostgreSQL) and converts every committed change into a Kafka event. A Kafka Connect sink connector then writes these events to Elasticsearch. This approach is decoupled, reliable, and handles all change types without any application code changes.

DB-to-ES Sync Patterns



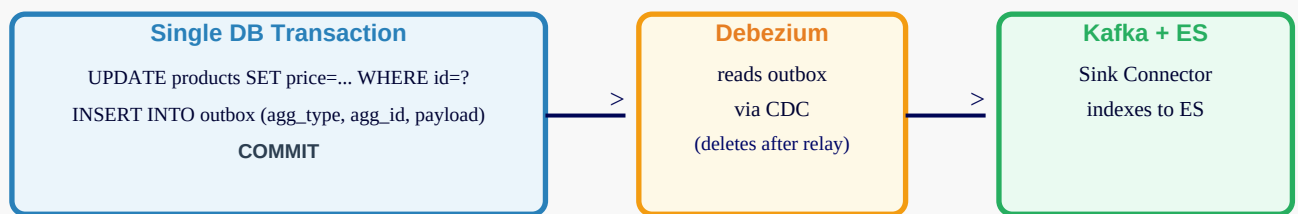
Debezium CDC Pipeline

Debezium CDC Pipeline



Transactional Outbox Pattern

Transactional Outbox Pattern



Outbox guarantees atomicity: if the DB transaction succeeds, the event will be published

No dual-write race condition. Eventual consistency with strong durability guarantees.

Key Definitions

CDC (Change Data Capture)	Technique of capturing row-level changes from DB transaction logs
Debezium	Open-source CDC platform built on Kafka Connect; supports MySQL, Pg, MongoDB, etc.
binlog	MySQL binary log: sequential record of all committed transactions
WAL (Write-Ahead Log)	PostgreSQL transaction log read by Debezium to capture changes
Kafka Connect	Kafka framework for streaming data between systems using connectors
Source connector	Reads data FROM a source system into Kafka (e.g., Debezium MySQL connector)
Sink connector	Reads FROM Kafka and writes to a target (e.g., Elasticsearch sink)
outbox pattern	Atomically write domain event to outbox table in same DB transaction
dual-write	Application writes to both DB and ES; prone to partial failure inconsistency
snapshot	Debezium initial read of all existing rows before tailing the log

LSN/offset	Log Sequence Number/offset tracking Debezium's position in the log
tombstone event	Kafka message with null value representing a deleted record
SMT (Single Message Transform)	Kafka Connect transform applied to messages in the pipeline
eventually consistent	ES will catch up to DB; momentary lag is acceptable but bounded
at-least-once delivery	Kafka + Debezium guarantees delivery but may duplicate; use idempotent indexing

Sync Pattern Trade-offs

Pattern	Handles Deletes?	Consistency	Complexity	App Change?	Latency
Dual-write	Yes	Weak (race)	Low	Yes	ms
Polling/timestamp	No	Eventual	Low	Yes	seconds
CDC (Debezium)	Yes	Strong eventual	Medium	No	sub-second
Outbox + CDC	Yes	Strong eventual	High	Yes	sub-second

Code Examples

1. Debezium MySQL connector configuration

```
// POST http://localhost:8083/connectors { "name": "mysql-products-connector", "config": {
"connector.class": "io.debezium.connector.mysql.MySqlConnector", "database.hostname": "mysql",
"database.port": "3306", "database.user": "debezium", "database.password": "${DB_PASSWORD}",
"database.server.id": "184054", "database.server.name": "mydb", "database.include.list":
"ecommerce", "table.include.list": "ecommerce.products,ecommerce.categories",
"database.history.kafka.bootstrap.servers": "kafka:9092", "database.history.kafka.topic":
"schema-changes.ecommerce", "include.schema.changes": "true", "snapshot.mode": "initial", //
take initial snapshot, then tail "decimal.handling.mode": "string", // avoid BigDecimal
precision issues "time.precision.mode": "connect", "topic.prefix": "cdc" // topic:
cdc.ecommerce.products } }
```

2. Elasticsearch Sink Connector configuration

```
// POST http://localhost:8083/connectors { "name": "es-products-sink", "config": {
"connector.class": "io.confluent.connect.elasticsearch.ElasticsearchSinkConnector",
"tasks.max": "1", "topics": "cdc.ecommerce.products", "connection.url":
"http://elasticsearch:9200", "type.name": "_doc", "key.ignore": "false", // use Kafka key as ES
_id "schema.ignore": "true", "behavior.on.null.values": "delete", // null value = delete in ES
"transforms": "ExtractAfter,AddTimestamp", "transforms.ExtractAfter.type":
"org.apache.kafka.connect.transforms.ExtractField$Value", "transforms.ExtractAfter.field":
"after", // use "after" field from CDC event "transforms.AddTimestamp.type":
"org.apache.kafka.connect.transforms.InsertField$Value",
"transforms.AddTimestamp.timestamp.field": "indexed_at" } }
```

3. CDC event structure from Debezium

```
// Debezium change event structure for an UPDATE: { "schema": { ... }, "payload": { "before": {
"id": "prod-123", "name": "Running Shoes", "price": "89.99", "stock": 50 }, "after": { "id":
```

```
"prod-123", "name": "Running Shoes Pro", "price": "99.99", "stock": 45 }, "source": { "db":
"ecommerce", "table": "products", "ts_ms": 1700000000000, "gtid": "...", "pos": 12345 }, "op":
"u", // c=create, u=update, d=delete, r=read(snapshot) "ts_ms": 1700000000100 } } // For delete
(op: "d"): // after = null, before = { ...deleted row data... } // Kafka sink sees null value
-> deletes from ES
```

4. Custom Kafka consumer to sync ES (Java)

```
@Component public class ProductCDCConsumer { private final ElasticsearchOperations esOps;
private final ObjectMapper objectMapper; @KafkaListener( topics = "cdc.ecommerce.products",
groupId = "es-sync-group", containerFactory = "cdcKafkaListenerContainerFactory" ) public void
consume(ConsumerRecord<String, String> record) { try { String key = record.key(); // product ID
if (record.value() == null) { // Tombstone = delete esOps.delete(key,
IndexCoordinates.of("products")); return; } JsonNode event =
objectMapper.readTree(record.value()); String op = event.path("payload").path("op").asText();
JsonNode after = event.path("payload").path("after"); switch (op) { case "c", "r", "u" -> {
ProductDocument doc = objectMapper.treeToValue(after, ProductDocument.class); esOps.save(doc);
} case "d" -> { esOps.delete(key, IndexCoordinates.of("products")); } default ->
log.warn("Unknown op: {}", op); } } catch (Exception e) { log.error("Failed to sync record
key={},", record.key(), e); // Re-throw to trigger retry via @RetryableTopic or DLT throw new
RuntimeException(e); } } }
```

5. Transactional Outbox pattern implementation

```
// Step 1: Create outbox table CREATE TABLE outbox ( id UUID PRIMARY KEY DEFAULT
gen_random_uuid(), aggregate_type VARCHAR(255) NOT NULL, -- "Product" aggregate_id
VARCHAR(255) NOT NULL, -- product ID event_type VARCHAR(255) NOT NULL, -- "ProductUpdated"
payload JSONB NOT NULL, created_at TIMESTAMPTZ DEFAULT NOW() ); // Step 2: Write to both
product table and outbox in one transaction @Transactional public Product updateProduct(String
id, UpdateProductCommand cmd) { Product product = productRepository.findById(id)
.orElseThrow(() -> new ProductNotFoundException(id)); product.setPrice(cmd.getPrice());
product.setName(cmd.getName()); productRepository.save(product); // Write to outbox in SAME
transaction outboxRepository.save(OutboxEvent.builder() .aggregateType("Product")
.aggregateId(id) .eventType("ProductUpdated") .payload(objectMapper.valueToTree(product))
.build()); return product; } // Step 3: Debezium reads outbox table via CDC // Configure
debezium to read only the `outbox` table // Use Debezium Outbox Event Router SMT to route to
topic by aggregate_type
```

6. Handling reindex (full sync) via Kafka

```
// Full reindex: read all rows and republish to Kafka topic @Component public class
FullReindexJob { private final ProductRepository productRepo; private final
KafkaTemplate<String, ProductDocument> kafkaTemplate; @Scheduled(cron = "0 0 2 * * SUN") //
weekly at 2 AM public void fullReindex() { log.info("Starting full reindex"); String topic =
"product.reindex"; // Stream all products in batches to avoid OOM
productRepo.findAll(Sort.by("id")) .forEach(product -> kafkaTemplate.send(topic,
product.getId(), new ProductDocument(product))); log.info("Full reindex published to Kafka");
// ES sink connector picks up and reindexes } } // For large tables, use cursor-based batching:
String lastId = null; do { List<Product> batch =
productRepo.findTop1000ByIdGreaterThanOrderBy(lastId); batch.forEach(p ->
kafkaTemplate.send(topic, p.getId(), new ProductDocument(p))); lastId = batch.isEmpty() ? null
: batch.get(batch.size()-1).getId(); } while (lastId != null);
```

Interview Questions & Answers

Q1. Why is dual-write problematic for DB-to-ES synchronization?

Dual-write means the application writes to both DB and ES in the same code path. Problems: (1) If the DB write succeeds but ES write fails, the two are now inconsistent — no automatic retry. (2) If ES write succeeds but DB crashes before commit, ES has data the DB rolled back. (3) No way to achieve distributed atomicity without two-phase commit (which is rarely used). (4) Race conditions with concurrent updates — ES may receive updates out of order. Use CDC instead for reliable sync.

Q2. What is CDC and how does Debezium implement it?

Change Data Capture (CDC) captures row-level database changes by reading the transaction log, not by polling the tables. Debezium is an open-source CDC platform built as Kafka Connect source connectors. It reads MySQL's binlog, PostgreSQL's WAL, MongoDB's oplog, etc. For each committed transaction, it publishes a structured event to a Kafka topic containing the operation type (c/u/d), before/after images of the row, and metadata (DB, table, timestamp).

Q3. What is a binlog and what privileges does Debezium need?

MySQL's binary log (binlog) is a sequential log of all committed DML operations. Debezium reads it to detect changes. Requirements: (1) `binlog_format=ROW` (row-level logging), (2) `binlog_row_image=FULL` (log complete before/after row images). Debezium MySQL user needs: `REPLICATION SLAVE`, `REPLICATION CLIENT`, `SELECT` (for snapshots), `RELOAD`, `LOCK TABLES`, `SHOW DATABASES`. Never use the root account.

Q4. How does Debezium handle snapshots?

On first startup (or when configured), Debezium performs a snapshot: it reads all existing rows from the configured tables and publishes them as ``r`` (read) events, then transitions to tailing the binlog/WAL for incremental changes. `snapshot.mode` options: ``initial`` (default — snapshot then tail), ``never`` (skip snapshot, start tailing from current position), ``always`` (re-snapshot every restart), ``schema_only`` (capture schema changes only, no data). Snapshots can take a long time for large tables.

Q5. What is a tombstone event in Kafka and why is it important for ES sync?

A tombstone is a Kafka message with a null value and the same key as the deleted record. When Debezium detects a delete operation, it publishes: (1) a delete event (op: 'd', after: null), and (2) a null-value tombstone to enable Kafka log compaction to clean up the key. The ES sink connector interprets a null value as a delete operation in ES. Without tombstones, Kafka's compacted topics would retain the key forever.

Q6. What is Kafka log compaction and why does it matter for CDC topics?

Log compaction retains only the LATEST message per key in a Kafka topic, discarding older versions. For CDC topics, this means after compaction, only the latest state of each record is retained (not the full change history). If ES falls behind and needs to re-consume, it sees only the latest state per record — still correct for sync but loses history. Enable compaction on CDC topics for space efficiency, especially for frequently-updated records.

Q7. Explain the Transactional Outbox Pattern.

The outbox pattern solves the dual-write problem with a guarantee: the application writes its domain event to an ``outbox`` table in the SAME database transaction as the domain object update. Either both succeed (commit) or both fail (rollback). Debezium then reads the outbox table via CDC and publishes events to Kafka. Since Debezium reads the DB log, not the application, events are published exactly when committed. This achieves atomicity without distributed transactions.

Q8. What is the Debezium Outbox Event Router SMT?

SMT = Single Message Transform. The Outbox Event Router is a Debezium-provided Kafka Connect transform that reads CDC events from the outbox table and routes them to different Kafka topics based on the `aggregate_type` column. Example: ProductUpdated events go to 'products' topic, OrderCreated events go to 'orders' topic. It also flattens the event structure to just the payload field and sets the Kafka key to the `aggregate_id`.

Q9. How do you handle schema changes in the CDC pipeline?

Debezium captures DDL changes (ALTER TABLE) and publishes them to a schema change topic (`database.history.kafka.topic`). It also updates its internal schema registry. Handling: (1) Add-only column changes are handled automatically — new column appears in events. (2) Column renames/drops require updating the ES mapping and handling the schema evolution in the sink connector. (3) For ES, adding a new field is safe; changing a field type requires reindexing. Use `schema.history.internal` connector to maintain state.

Q10. What are the delivery guarantees with Debezium + Kafka?

Debezium provides at-least-once delivery — in failure/restart scenarios, some events may be re-delivered. Kafka itself provides at-least-once delivery by default. The ES sink connector handles duplicates by using the record key as ES document ID (idempotent write: indexing the same document twice with the same ID just overwrites). Deletes must be idempotent too — deleting a non-existent document is a no-op in ES. The result is effectively exactly-once in ES for most use cases.

Q11. How do you monitor Kafka Connect connector lag?

Monitor: (1) Kafka consumer group lag for the sink connector group — how many events are behind. (2) Kafka Connect REST API: GET `/connectors/{name}/status` to check connector state and any errors. (3) JMX metrics from Kafka Connect. (4) Debezium-specific metrics via JMX: `MillisecondsBehindSource`, `NumberOfCommittedTransactions`, `LastEventTimestamp`. Alert when lag exceeds a threshold (e.g., > 5 minutes behind). High lag indicates: ES is slow to ingest, or connector is unhealthy.

Q12. How do you handle a connector failure and resume without data loss?

Debezium stores its current binlog/WAL position (offset) in Kafka (offset storage topic). On restart, it reads the saved offset and resumes from that exact position. Committed events are never lost. For pauses > binlog retention period (risk: binlog purged): (1) Rebuild from snapshot (increase `snapshot.mode=always`) or (2) Increase MySQL `binlog_expire_logs_seconds`. Monitor binlog age vs. connector lag. For Kafka Connect task failures, the Connect framework retries automatically.

Q13. How would you implement filtering in the CDC pipeline — only sync some columns to ES?

Options: (1) Debezium `column.include.list` / `column.exclude.list` — only capture specified columns in the CDC event. (2) Kafka Connect SMT `ReplaceField` to drop fields before the ES sink connector receives them. (3) Custom transform or KSQL/ksqlDB to project only needed fields. (4) In a custom Kafka consumer (Java), manually construct the ES document from the CDC event, selecting only the needed fields. Column filtering at the Debezium level is most efficient (less data in Kafka).

Q14. What is the difference between `table.include.list` and `database.include.list` in Debezium?

Both are whitelist filters. `database.include.list` limits which databases Debezium monitors. `table.include.list` limits which tables within monitored databases produce events (format: `'dbname.tablename'`). Best practice: always specify both to minimize volume and avoid capturing sensitive tables (user credentials, payment data). Not specifying them means Debezium tracks ALL tables and databases — unnecessary overhead and security risk.

Q15. How do you handle large binary/blob fields in CDC events?

Binary/blob fields appear in CDC events as base64-encoded strings, bloating Kafka messages. Options: (1) Exclude blob columns from CDC with `column.exclude.list`. (2) Store blobs in object storage (S3) and only store the URL in ES. (3) Use Debezium's `binary.handling.mode` to control encoding (bytes vs. base64 vs. hex). (4) For ES: map binary fields with `binary` type (stored but not searchable) or exclude them entirely from the ES document.

Q16. What are common causes of CDC consumer lag?

1) ES indexing throughput lower than DB write rate — increase ES shards, bulk batch size. 2) ES rejected requests (429 Too Many Requests) due to queue full — tune `queue_size` and `thread_pool` settings. 3) Connector task errors causing retries — fix the error first. 4) Kafka broker throughput bottleneck — increase partition count. 5) Snapshot taking a very long time — expected for initial sync of large tables. 6) Network issues between Kafka Connect and ES.

Q17. How do you keep ES in sync when the primary DB is MongoDB?

Debezium supports MongoDB via the MongoDB connector, reading the oplog (older) or change streams (MongoDB 3.6+, recommended). Change streams emit create/update/delete/replace events. Configuration similar to MySQL connector with `connector.class: io.debezium.connector.mongodb.MongoDbConnector`. Key difference: MongoDB documents are already JSON, so the ES document structure maps more naturally. Handle `$set` partial updates: use `update_by_script` in ES sink or read the full document from MongoDB.

Q18. What is eventually consistent search and how do you communicate it to users?

Eventually consistent means ES may lag behind the DB by milliseconds to seconds. After a user creates/updates a record, they may not immediately see it in search results. Best practices: (1) After write, show the record from DB directly (not search), redirect to a detail page. (2) Display a 'Search results may take a moment to update' notice near search on write-intensive pages. (3) In test environments, wait for ES refresh between write and search assertions. (4) Tune `refresh_interval` lower (e.g., 1s) for critical search paths needing near-real-time visibility.

Q19. How do you validate ES-DB consistency in production?

Implement a periodic reconciliation job: (1) Query DB for all record IDs and their `updated_at` timestamps. (2) Query ES for the same IDs and their `updated_at`. (3) Find mismatches: IDs in DB missing from ES, or ES timestamps behind DB by more than the acceptable lag. (4) Re-index the diverged records. This can be a scheduled batch job running daily. For mission-critical sync: run reconciliation hourly and alert on inconsistency count. Tools: Elasticsearch's multi-get API for efficient batch comparison.

Q20. What is a dead letter topic (DLT) and why is it important in CDC pipelines?

A dead letter topic (DLT) captures events that failed to process after all retries, preventing them from blocking the pipeline. In Kafka Connect, configure `errors.tolerance=all` and `errors.deadletterqueue.topic.name=dlt.es.products`. Failed events go to the DLT with error metadata headers. Without DLT: one bad event blocks the entire partition (poison pill). Monitor DLT for: malformed events, ES mapping conflicts, events with missing required fields. Implement alerts and tooling to replay or fix DLT events.